

CreativeMind Studio

Silent Killer - Backend, RAG, Memory & IBM AI Integration Lead

Concise Execution Brief

Primary Objective

Build the backend intelligence layer that turns a raw creative idea into a stored, IBM Granite-powered multi-agent debate and returns validated JSON for the frontend.

Mandatory IBM Requirement

- IBM Granite must power the real application workflow.
- IBM Bob may support development, but it cannot be the only IBM technology.
- The final demo should expose metadata such as **ai_engine: IBM Granite**.

Recommended Stack

Layer	Choice
Backend	Python + FastAPI + Pydantic
Database	Supabase/PostgreSQL or SQLAlchemy
AI	IBM Granite / watsonx.ai
Orchestration	LangGraph preferred; native state machine acceptable
Memory/RAG	Database memory first; curated vector knowledge later

Core Debate Flow

Raw Idea → Creative Director → Risk Critic → Technical/Market Agent → counter-response → Audience & Marketing → Ethical Auditor → Execution Planner → Synthesis Agent

Seven Required Agents

- Creative Director - originality and emotional direction
- Risk Critic - assumptions, weaknesses and risks
- Technical/Market Agent - feasibility, cost and market logic
- Audience Analyst - audience reactions and confusion
- Marketing Strategist - hook, positioning and distribution
- Execution Planner - MVP scope and roadmap
- Ethical & Brand-Safety Auditor - green/amber/red rating

Required API Surface

Endpoint	Purpose
GET /health	Backend, DB and Granite status
POST /api/projects	Create project
POST /api/projects/{id}/boardroom	Run Granite debate

Endpoint	Purpose
GET /api/projects/{id}/boardroom	Retrieve debate
POST /api/projects/{id}/follow-up	Add user context
POST /api/projects/{id}/audience-simulation	Persona simulation
POST /api/projects/{id}/scorecard	Scores with reasons
POST /api/projects/{id}/pivot	Recalculate scope and risks
POST /api/projects/{id}/refine	Generate refined idea
POST /api/projects/{id}/roadmap	Generate roadmap
POST /api/projects/{id}/production-package	Generate assets
GET /api/projects/{id}/report	Final combined JSON

Structured Output & Storage

- Return valid JSON only; validate with Pydantic.
- Include debate, strengths, weaknesses, scored dimensions with reasons, ethical audit, audience simulation, refined idea, roadmap and production package.
- Store projects, boardroom sessions, agent messages, scorecards and idea versions.
- Frontend should never parse raw model text.

Memory, RAG & Intelligence Layers

- **Session memory:** debate turns, follow-ups, accepted and rejected suggestions.
- **Long-term memory:** versions, score history, pivots and project decisions.
- **RAG:** small trusted library of creative strategy, storytelling, marketing and safety documents.
- **Trend Radar:** cached/stubbed current signals for MVP; never block the main flow.
- **Virality Twin:** begin with 30-50 curated examples, not large-scale scraping.

Reliability & Cost Control

- Input validation, timeout, retry and one JSON-repair attempt.
- Fallback only when Granite fails; record fallback activation.
- Cache repeated analyses and limit debate turns/tokens.
- Pivot should rerun only relevant agents, not the full boardroom.
- Never commit `.env` or IBM credentials.

First Sprint - Definition of Done

1. FastAPI app, health endpoint and database schema
2. Project-create endpoint and storage
3. One successful IBM Granite call
4. Creative Director, Risk Critic and Technical/Market Agent
5. At least two connected debate turns
6. Synthesis response validated by Pydantic
7. Stored/retrievable messages and session
8. FastAPI `/docs`, `.env.example` and setup guide

Expected First Milestone

Raw idea → IBM Granite three-agent debate → synthesis → validated JSON → database storage → frontend-ready response.

Reply Before Starting

- Preferred backend/database stack
- IBM Granite access method
- LangGraph or native orchestration
- Proposed folder structure
- ETA for first Granite-powered boardroom endpoint
- Credentials or blockers needed